Лабораторна робота №3

Структурні шаблони

Мета роботи: навчитися реалізовувати структурні шаблони проєктування Адаптер, Декоратор, Міст, Компонувальник, Проксі, Легковаговик.

Термін виконання: 2 год (1 заняття)

Зміст роботи

Завдання №1

1. Створіть клас Logger, який буде мати методи Log(), Error(), Warn(), які вводять повідомлення в консоль різними кольорами (зеленим, червоним і оранжевим відповідно).

2. Створіть клас FileWriter з методами Write(), WriteLine().

3. За допомогою шаблону Адаптер створіть файловий логер.

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми

Лістинг програми:

Logger.cs:

```
using System;
using System.IO;
class Logger
{
    public void Log(string message)
    {
        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine("LOG: " + message);
        Console.ResetColor();
    }

    public void Error(string message)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine("ERROR: " + message);
        Console.ResetColor();
    }

    public void Warn(string message)
    {
```

```
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine("WARN: " + message);
        Console.ResetColor();
    }
}
```

FileWriter:

```
class FileWriter
{
    private string filePath;

    public FileWriter(string path)
    {
        filePath = path;
    }

    public void Write(string message)
    {
        File.AppendAllText(filePath, message);
    }

    public void WriteLine(string message)
    {
        File.AppendAllText(filePath, message + Environment.NewLine);
    }
}
```

FileLoggerAdapter:

```
class FileLoggerAdapter
{
    private FileWriter fileWriter;

    public FileLoggerAdapter(string path)
    {
        fileWriter = new FileWriter(path);
    }

    public void Log(string message)
    {
        fileWriter.WriteLine("LOG: " + message);
    }
```

```
    public void Error(string message)
    {
        fileWriter.WriteLine("ERROR: " + message);
    }

    public void Warn(string message)
    {
        fileWriter.WriteLine("WARN: " + message);
    }
}
```

Program:

```
class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        Logger consoleLogger = new Logger();
        consoleLogger.Log("Це звичайне повідомлення.");
        consoleLogger.Warn("Це попередження.");
        consoleLogger.Error("Це помилка!");

        Console.WriteLine();
        FileLoggerAdapter fileLogger = new FileLoggerAdapter("log.txt");
        fileLogger.Log("Файл: звичайне повідомлення");
        fileLogger.Warn("Файл: попередження");
        fileLogger.Error("Файл: помилка");

        Console.WriteLine("Повідомлення записані в log.txt");
    }
}
```

Результат виконання програми:

```
Microsoft Visual Studio Debug Console
LOG: Це звичайне повідомлення.
WARN: Це попередження.
ERROR: Це помилка!

Повідомлення записані в log.txt

C:\Users\Masha\source\repos\kpz3\kpz3\bin\Debug
To automatically close the console when debuggi
le when debugging stops.
Press any key to close this window . . .
```

Рис1. Результат виконання програми.

Завдання 2.

1. Ви розробляєте РПГ гру. Створіть класи героїв Warrior, Mage, Palladin.

2. Для героїв створіть інвентар (одяг, зброю, артефакти), який може підходити будь-якому типу героїв, у вигляді декораторів.

3. Важливою вимогою є можливість використання декількох екземплярів інвентаря на герої одночасно.

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

```
class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        Hero warrior = new Warrior("Артур");
        Console.WriteLine("=== Базові характеристики героя ===");
        warrior.ShowStats();
        Console.WriteLine();
        Hero equippedWarrior = new Armor(new Weapon(new Artifact(warrior)));

        Console.WriteLine("=== Герой з інвентарем ===");
        equippedWarrior.ShowStats();

        Console.WriteLine();
        Console.WriteLine("=== Інший приклад: Mage з артефактом та бронею ===");
```

```
        Hero mage = new Mage("Мерлін");
        Hero equippedMage = new Armor(new Artifact(mage));
        equippedMage.ShowStats();
    }
}
```

Базові класи героїв:

```
using System;
abstract class Hero
{
    public string Name { get; set; }

    public Hero(string name)
    {
        Name = name;
    }

    public abstract void ShowStats();
}
class Warrior : Hero
{
    public Warrior(string name) : base(name) { }

    public override void ShowStats()
    {
        Console.WriteLine($"{Name} – Warrior: сила 10, магія 2, здоров'я 100");
    }
}
class Mage : Hero
{
    public Mage(string name) : base(name) { }

    public override void ShowStats()
    {
        Console.WriteLine($"{Name} – Mage: сила 3, магія 12, здоров'я 80");
    }
}
class Palladin : Hero
{
    public Palladin(string name) : base(name) { }

    public override void ShowStats()
    {
        Console.WriteLine($"{Name} – Palladin: сила 7, магія 5, здоров'я 90");
```

```
    }
}
```

Абстрактний інвентар:

```
abstract class InventoryDecorator : Hero
{
    protected Hero hero;

    public InventoryDecorator(Hero hero) : base(hero.Name)
    {
        this.hero = hero;
    }

    public override void ShowStats()
    {
        hero.ShowStats();
    }
}
```

Конкретні декоратори:

```
class Armor : InventoryDecorator
{
    public Armor(Hero hero) : base(hero) { }

    public override void ShowStats()
    {
        hero.ShowStats();
        Console.WriteLine(" + Носить броню: +20 до захисту");
    }
}

class Weapon : InventoryDecorator
{
    public Weapon(Hero hero) : base(hero) { }

    public override void ShowStats()
    {
        hero.ShowStats();
        Console.WriteLine(" + Озброєний зброєю: +15 до атаки");
    }
}

class Artifact : InventoryDecorator
```

```
    {
        public Artifact(Hero hero) : base(hero) { }

        public override void ShowStats()
        {
            hero.ShowStats();
            Console.WriteLine(" + Має артефакт: +10 до магії");
        }
    }
```

Результат виконання програми:

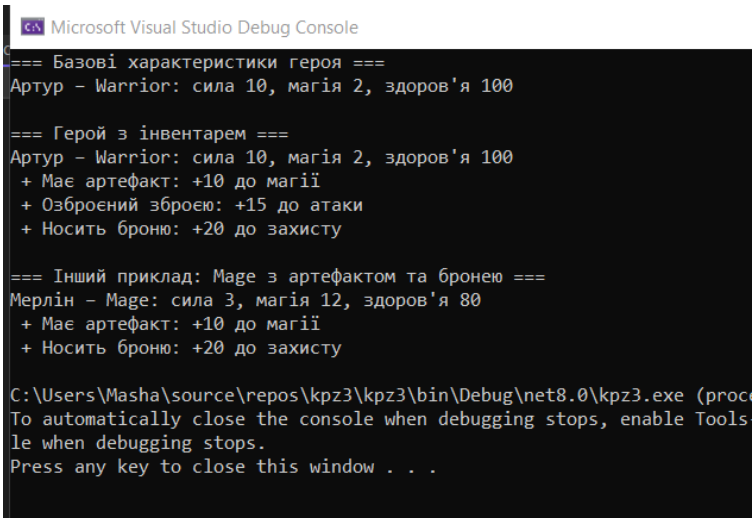

Рис2. Результат виконання програми.

Завдання 3.

1. Ви працюєте над графічним редактором. Створіть базовий клас Shape.

2. Створіть дочірні до Shape класи, Circle, Square, Triangle.

3. За допомогою шаблону Міст додайте можливість рендерингу кожної з фігур як векторної або растрової графіки (вивівши відповідне повідомлення у консоль, наприклад "Drawing Triangle as pixels").

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми

Лістинг програми:

Shape:

```
using System;

abstract class Shape
{
    protected IRenderer renderer;
```

```
    public Shape(IRenderer renderer)
    {
        this.renderer = renderer;
    }

    public abstract void Draw();
     }
```

Інтерфейс рендерера:

```
interface IRenderer
{
    void RenderCircle();
    void RenderSquare();
    void RenderTriangle();
     }
```

Конкретні рендерери:

```
class VectorRenderer : IRenderer
{
    public void RenderCircle()
    {
        Console.WriteLine("Drawing Circle as vectors");
    }

    public void RenderSquare()
    {
        Console.WriteLine("Drawing Square as vectors");
    }

    public void RenderTriangle()
    {
        Console.WriteLine("Drawing Triangle as vectors");
    }
}

class RasterRenderer : IRenderer
{
    public void RenderCircle()
    {
        Console.WriteLine("Drawing Circle as pixels");
    }

    public void RenderSquare()
    {
        Console.WriteLine("Drawing Square as pixels");
    }

    public void RenderTriangle()
    {
        Console.WriteLine("Drawing Triangle as pixels");
    }
     }
```

Конкретні фігури:

```
class Circle : Shape
{
    public Circle(IRenderer renderer) : base(renderer) { }

    public override void Draw()
    {
        renderer.RenderCircle();
    }
}
```

```
class Square : Shape
{
    public Square(IRenderer renderer) : base(renderer) { }

    public override void Draw()
    {
        renderer.RenderSquare();
    }
}

class Triangle : Shape
{
    public Triangle(IRenderer renderer) : base(renderer) { }

    public override void Draw()
    {
        renderer.RenderTriangle();
    }
 }
```

Демонстрація:

```
class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;
        IRenderer vector = new VectorRenderer();
        IRenderer raster = new RasterRenderer();

        Shape circleVector = new Circle(vector);
        Shape squareVector = new Square(vector);
        Shape triangleVector = new Triangle(vector);

        Console.WriteLine("=== Векторна графіка ===");
        circleVector.Draw();
        squareVector.Draw();
        triangleVector.Draw();

        Console.WriteLine();

        Shape circleRaster = new Circle(raster);
        Shape squareRaster = new Square(raster);
        Shape triangleRaster = new Triangle(raster);

        Console.WriteLine("=== Растрова графіка ===");
        circleRaster.Draw();
        squareRaster.Draw();
        triangleRaster.Draw();
    }
 }
```

Результат виконання програми:

```
Microsoft Visual Studio Debug Console
=== Векторна графіка ===
Drawing Circle as vectors
Drawing Square as vectors
Drawing Triangle as vectors

=== Растрова графіка ===
Drawing Circle as pixels
Drawing Square as pixels
Drawing Triangle as pixels

C:\Users\Masha\source\repos\kpz3\kpz3\bin\Debug\net8.
To automatically close the console when debugging sto
le when debugging stops.
Press any key to close this window . . .
```

Рис3. Результат виконання програми.

Завдання 4.

1. Створіть клас SmartTextReader, який вміє читати вміст текстового файлу і перетворювати його на двомірний масив якому зовнішній масив відповідає рядкам тексту, а вкладені масиви відповідають символам у відповідному рядку.

2. Створіть проксі для SmartTextReader з логуванням SmartTextChecker, який буде виводити інформацію про успішне відкриття, прочитання і закриття файлу, а також буде виводити загальну кількість рядків і символів у прочитаному тексті.

3. Створіть проксі для SmartTextReader з обмеженням доступу до певних файлів SmartTextReaderLocker. Цей клас в конструкторі приймає регулярний вираз, по якому лімітується доступ до певної групи файлів. Якщо клієнт викликатиме метод для прочитання такого лімітованого файлу, замість прочитання файлу в консоль має виводитися повідомлення “Access denied!”.

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

SmartTextReader:

```
using System;
using System.IO;
using System.Text.RegularExpressions;
```

```
class SmartTextReader
{
    private string filePath;

    public SmartTextReader(string path)
    {
        filePath = path;
    }

    public char[][] ReadFile()
    {
        string[] lines = File.ReadAllLines(filePath);
        char[][] result = new char[lines.Length][];
        for (int i = 0; i < lines.Length; i++)
        {
            result[i] = lines[i].ToCharArray();
        }
        return result;
    }

    public string FileName => filePath;
     }
```

SmartTextChecker:

```
class SmartTextChecker
{
    private SmartTextReader reader;

    public SmartTextChecker(SmartTextReader reader)
    {
        this.reader = reader;
    }

    public char[][] ReadFile()
    {
        Console.WriteLine($"Opening file: {reader.FileName} ...");
        char[][] content = reader.ReadFile();
        Console.WriteLine("File read successfully!");
        int totalLines = content.Length;
        int totalChars = 0;
        foreach (var line in content)
        {
            totalChars += line.Length;
        }
        Console.WriteLine($"Lines: {totalLines}, Characters: {totalChars}");
        Console.WriteLine("Closing file...");
        return content;
    }
     }
```

SmartTextReaderLocker:

```
using System.Text.RegularExpressions;

class SmartTextReaderLocker
{
    private SmartTextReader reader;
    private Regex accessRegex;

    public SmartTextReaderLocker(SmartTextReader reader, string pattern)
    {
        this.reader = reader;
        accessRegex = new Regex(pattern, RegexOptions.IgnoreCase);
    }
```

```
    public char[][] ReadFile()
    {
        if (!accessRegex.IsMatch(reader.FileName))
        {
            Console.WriteLine("Access denied!");
            return null;
        }

        return reader.ReadFile();
    }
     }
```

Main:

```
class Program
{
    static void Main(string[] args)
    {

        string filePath = "test.txt";

        Console.WriteLine("=== SmartTextChecker (логування) ===");
        SmartTextReader reader = new SmartTextReader(filePath);
        SmartTextChecker checker = new SmartTextChecker(reader);
        char[][] content = checker.ReadFile();

        Console.WriteLine();
        Console.WriteLine("=== SmartTextReaderLocker (обмеження доступу) ===");
        SmartTextReaderLocker locker = new SmartTextReaderLocker(reader, @"\.txt$");
        char[][] allowed = locker.ReadFile();
        SmartTextReader forbiddenReader = new SmartTextReader("secret.dat");
        SmartTextReaderLocker locker2 = new SmartTextReaderLocker(forbiddenReader,
@"\.txt$");
        char[][] denied = locker2.ReadFile();
    }
     }
```

Результат виконання програми:

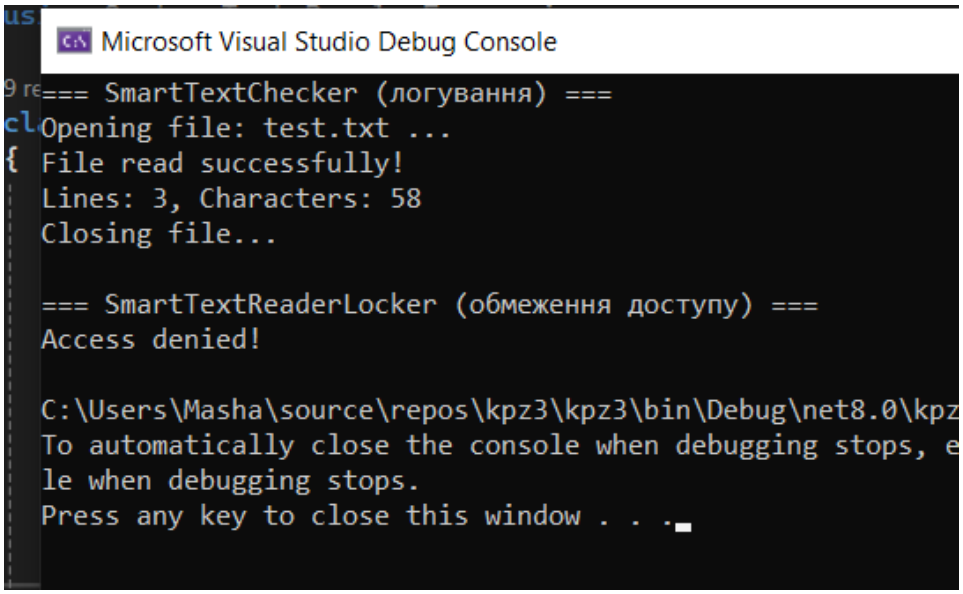

Рис4. Результат виконання програми.

Завдання 5.

1. Вам потрібно створити власну мову розмітки LightHTML.

2. Кожен елемент розмітки має наслідувати клас LightNode.

3. Створіть два дочірніх класи від LightNode: LightElementNode, LightTextNode.

4. LightTextNode може містити лише текст.

5. LightElementNode може містити будь-які LightNode. LightElementNode повинен мати інформацію про назву тега, його тип відображення (блочний чи рядковий), тип закриття (одиничний тег, як <img/> чи з закриваючим тегом) список CSS класів, кількість дочірніх елементів, а також має бути можливість виводити на екран його outerHTML і innerHTML.

6. За допомогою своєї мови розмітки виведіть в консоль елемент сторінки на Ваш вибір (наприклад якусь таблицю, список тощо).

7. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

LightNode:

```
using System;
using System.Collections.Generic;
using System.Text;

abstract class LightNode
{
    public abstract string OuterHTML { get; }
    public abstract string InnerHTML { get; }
     }
```

LightTextNode:

```
class LightTextNode : LightNode
{
    public string Text { get; set; }

    public LightTextNode(string text)
    {
        Text = text;
    }

    public override string OuterHTML => Text;
    public override string InnerHTML => Text;
     }
```

LightElementNode:

```
using System.Text;

enum DisplayType { Block, Inline }
enum CloseType { SelfClosing, Normal }
```

```
class LightElementNode : LightNode
{
    public string TagName { get; set; }
    public DisplayType Display { get; set; }
    public CloseType Close { get; set; }
    public List<string> Classes { get; set; } = new List<string>();
    public List<LightNode> Children { get; set; } = new List<LightNode>();

    public LightElementNode(string tagName, DisplayType display = DisplayType.Block,
CloseType close = CloseType.Normal)
    {
        TagName = tagName;
        Display = display;
        Close = close;
    }

    public void AddChild(LightNode child)
    {
        Children.Add(child);
    }

    public int ChildCount => Children.Count;

    public override string InnerHTML
    {
        get
        {
            StringBuilder sb = new StringBuilder();
            foreach (var child in Children)
            {
                sb.Append(child.OuterHTML);
            }
            return sb.ToString();
        }
    }

    public override string OuterHTML
    {
        get
        {
            string classAttr = Classes.Count > 0 ? $" class=\"{string.Join(" ",
Classes)}\"" : "";
            if (Close == CloseType.SelfClosing)
            {
                return $"<{TagName}{classAttr}/>";
            }
            else
            {
                return $"<{TagName}{classAttr}>{InnerHTML}</{TagName}>";
            }
        }
    }
     }
```

Program:

```
class Program
{
    static void Main(string[] args)
    {
        LightElementNode ul = new LightElementNode("ul", DisplayType.Block);
        ul.Classes.Add("list");

        LightElementNode li1 = new LightElementNode("li", DisplayType.Block);
```

```
        li1.AddChild(new LightTextNode("Перший елемент"));
        LightElementNode li2 = new LightElementNode("li", DisplayType.Block);
        li2.AddChild(new LightTextNode("Другий елемент"));
        LightElementNode li3 = new LightElementNode("li", DisplayType.Block);
        li3.AddChild(new LightTextNode("Третій елемент"));

        ul.AddChild(li1);
        ul.AddChild(li2);
        ul.AddChild(li3);

        Console.WriteLine("=== OuterHTML ===");
        Console.WriteLine(ul.OuterHTML);

        Console.WriteLine("\n=== InnerHTML ===");
        Console.WriteLine(ul.InnerHTML);
    }
}
```

Результат виконання програми:

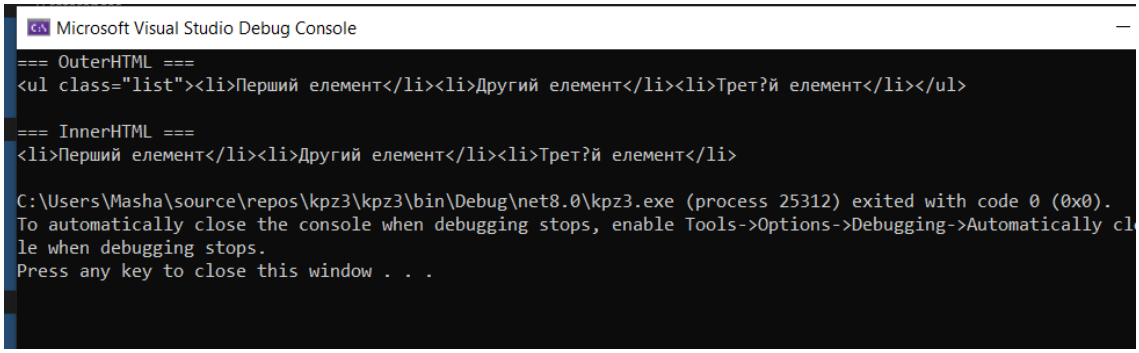

Рис5. Результат виконання програми.

Завдання 6.

1. За допомогою свого LightHTML з завдання 1 перетворіть текст книги в HTML верстку за такими правилами:

a. Перший рядок має бути елементом <h1>

b. Якщо в рядку менше 20 символів - це має бути елемент <h2>

c. Якщо рядок починається з пробільного символу - це має бути <blockquote>.

d. В будь-якому іншому випадку - елемент <p>

2. Покажіть скільки займає все дерево Вашої верстки, коли воно повністю утримується в пам’яті процесу.

3. Використайте Легковаговик на Ваших класах HTML елементів, щоб зменшити споживання пам’яті.

4. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Лістинг програми:

LightElementNode.cs:

```
using System;
using System.Collections.Generic;
using System.Text;

abstract class LightNode
{
    public abstract string OuterHTML { get; }
    public abstract string InnerHTML { get; }
}
class LightTextNode : LightNode
{
    public string Text { get; set; }

    public LightTextNode(string text)
    {
        Text = text;
    }

    public override string OuterHTML => Text;
    public override string InnerHTML => Text;
}

class TagFlyweight
{
    private static Dictionary<string, LightElementNode> _flyweights = new
Dictionary<string, LightElementNode>();

    public static LightElementNode GetTag(string tagName)
    {
        if (!_flyweights.ContainsKey(tagName))
        {
            _flyweights[tagName] = new LightElementNode(tagName);
        }
        return _flyweights[tagName];
    }
}

class LightElementNode : LightNode
{
    public string TagName { get; set; }
    public List<string> Classes { get; set; } = new List<string>();
    public List<LightNode> Children { get; set; } = new List<LightNode>();

    public LightElementNode(string tagName)
    {
        TagName = tagName;
    }

    public void AddChild(LightNode child)
    {
        Children.Add(child);
    }

    public int ChildCount => Children.Count;

    public override string InnerHTML
    {
        get
        {
            StringBuilder sb = new StringBuilder();
            foreach (var child in Children)
            {
                sb.Append(child.OuterHTML);
```

```
            }
            return sb.ToString();
        }
    }

    public override string OuterHTML
    {
        get
        {
            string classAttr = Classes.Count > 0 ? $" class=\"{string.Join(" ",
Classes)}\"" : "";
            return $"<{TagName}{classAttr}>{InnerHTML}</{TagName}>";
        }
    }
 }
```

BookParser.cs:

```
class BookParser
{
    public static LightElementNode ParseText(string[] lines)
    {
        LightElementNode root = new LightElementNode("div");
        for (int i = 0; i < lines.Length; i++)
        {
            string line = lines[i];
            LightElementNode tagNode;

            if (i == 0)
            {
                tagNode = new LightElementNode("h1");
            }
            else if (line.Trim().Length < 20)
            {
                tagNode = new LightElementNode("h2");
            }
            else if (line.StartsWith(" "))
            {
                tagNode = new LightElementNode("blockquote");
            }
            else
            {
                tagNode = new LightElementNode("p");
            }

            tagNode.AddChild(new LightTextNode(line));
            root.AddChild(tagNode);
        }

        return root;
    }
 }
```

Program:

```
class Program
{
    static void Main(string[] args)
    {
        string[] bookLines = new string[]
        {
            "Моя книга",
            "Вступ",
            " Це перший абзац, який починається з пробілу.",
            "Другий рядок звичайний.",
            "Коротко",
```

```
            " Ще один цитатний абзац."
        };

        Console.WriteLine("=== Створюємо HTML дерево ===");

        long memBefore = GC.GetTotalMemory(true);
        LightElementNode htmlTree = BookParser.ParseText(bookLines);
        long memAfter = GC.GetTotalMemory(true);

        Console.WriteLine("=== OuterHTML повного дерева ===");
        Console.WriteLine(htmlTree.OuterHTML);

        Console.WriteLine();
        Console.WriteLine($"Зайнята пам'ять приблизно: {memAfter - memBefore} байт");

        Console.WriteLine();
        Console.WriteLine($"Кількість дочірніх елементів кореня: {htmlTree.ChildCount}");
    }
    }
```

Результат виконання програми:

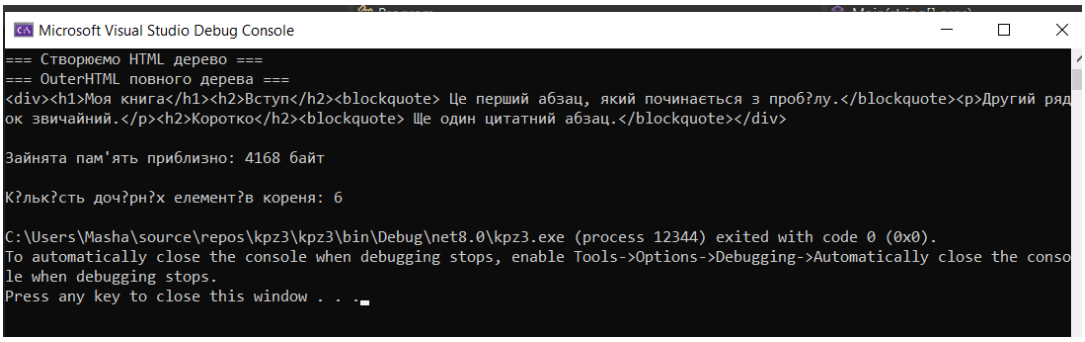

Рис6. Результат виконання програми.

Висновки: навчилася реалізовувати структурні шаблони проєктування Адаптер, Декоратор, Міст, Компонувальник, Проксі, Легковаговик.